

Viva Q&A: the project and DVC

Short answers to the questions a teacher is likely to ask, grouped by topic. Section 1 covers every question listed in the course PDF (DS-4491, section 4.1).

1. The questions from the course PDF

What does `dvc repro` do? It runs the pipeline defined in `dvc.yaml` (prepare → train → evaluate). Before each stage it checks whether anything that stage depends on (data, code, parameters) has changed since the last run. If nothing changed, it skips the stage; if something changed, it reruns that stage and every stage after it. It then updates `dvc.lock` with the new hashes.

Why are you using DVC? Git is made for code, not for 300 MB of images and model files. DVC lets us version the datasets and models alongside the code, store them in remote storage (DagsHub), and make the whole pipeline reproducible with one command. Anyone can get exactly our data, model and results with `git clone` → `dvc pull` → `dvc repro`.

What is `dvc.yaml`? The pipeline definition. For each stage it lists the command to run (`cmd`), the files it depends on (`deps`), the parameters it uses (`params`), and the files it produces (`outs` , `metrics` , `plots`). Our three stages are `prepare` , `train` and `evaluate` .

What is `params.yaml` ? One file with every setting of the pipeline: which dataset (`dataset: red`), the prepare settings, the training settings (mode, epochs, image size, batch, seed) and the evaluation settings (confidence threshold). Each stage in `dvc.yaml` says which parameters it uses, so DVC knows which stages are affected when a value changes.

How does DVC know which stage to rerun? `dvc.lock` stores the MD5 hash of every dependency, parameter value and output from the last run. `dvc status` / `dvc repro` compute the current hashes and compare. A stage is out of date if any of its dependencies or parameters changed, or if one of its outputs is missing or modified. Stages after it are rerun too, because their input (that stage's output) changes.

What happens if the dataset changes? Its hash changes, so the `.dvc` pointer no longer matches. `dvc status` shows `prepare` as out of date, and `dvc repro` reruns `prepare`, then `train` and `evaluate` (they depend on `prepare`'s output). After that you `dvc add` / `commit` the new pointer and `dvc push` the new files. Demo: switch `dataset: red` to `dataset: green` and all three stages rerun.

What happens if a parameter changes? Only the stages that use that parameter, plus the stages after them, rerun. Example: changing `evaluate.conf` from 0.25 to 0.5 reruns only `evaluate` ; changing `dataset` reruns all three. `dvc params diff` shows what changed and `dvc metrics diff` shows how the results changed.

What is the difference between Git and DVC? Git versions the code and small text files (including the `.dvc` pointer files and `dvc.lock`). DVC versions the large files (datasets, models): it keeps

them in its cache and in remote storage, and puts only a small pointer with their hash into Git. Git → GitHub; DVC data → DagsHub. DVC works on top of Git, not instead of it.

What do `dvc push` and `dvc pull` do? `dvc push` uploads the data and models that the pointer files refer to from the local cache to the remote (DagsHub), only the files the remote doesn't have yet. `dvc pull` does the reverse: reads the pointers, downloads the matching files from the remote, and puts them in the workspace. Like `git push / git pull`, but for the large files.

How can another person reproduce your experiment?

```
git clone https://github.com/TonmoyBishwas/laser-detector-mlsd-.git
cd laser-detector-mlsd-
python -m venv venv ; venv\Scripts\activate
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu128
pip install -r requirements.txt
(set the DagsHub login, see README)
dvc pull      # get the exact datasets, models and outputs
dvc repro     # rerun the pipeline; same data + same code + same parameters = same results
```

We tested this on a fresh clone: `dvc pull` downloaded everything and `dvc status` reported "Data and pipelines are up to date".

We also did it on a second, very different machine, a MacBook Air M4 (no NVIDIA GPU):



```
git clone https://github.com/TonmoyBishwas/laser-detector-mlsd-.git
cd laser-detector-mlsd-
uv venv --python 3.12 .venv ; source .venv/bin/activate
uv pip install torch torchvision -r requirements.txt      # normal PyT
orch, has Apple GPU support
(set the DagsHub login)
dvc pull          # 5,408 files: both datasets, both models, pipeline out
puts
dvc repro         # runs on the Apple GPU (mps) in about 30 s
```

The results matched the PC (see section 5, "Results on the Mac").

2. DVC in more depth

What is inside a `.dvc` file? A few lines of text. For example
`data/raw/red.dvc` :



```
outs:
- md5: b452f8f1b6647097a534c5942772c2e8.dir
  size: 234929614
  nfiles: 3497
  path: red
```

The hash identifies the exact content of the folder (`.dir` means a folder: 3,497 files, about 235 MB). If even one image changes, the hash changes.

What is `dvc.lock`? The record of the last pipeline run: for every stage, the hash of each dependency, the parameter values used, and the hash of each output. It is committed to Git, so the exact state of every run is versioned. `dvc.yaml` is what *should* run; `dvc.lock` is what *did* run.

What is the DVC cache? The folder `.dvc/cache`, where DVC stores every version of every tracked file, named by its hash. The workspace file is a copy of (or link to) a cache entry. Because files are stored by hash, identical files are stored once.

What is a DVC remote? Storage outside your computer where the cache is uploaded: here DagsHub
(<https://dagshub.com/ttonmoy46/laser-detector-mlsd-.dvc>). It could also be S3, Google Drive, Azure, SSH or a shared folder. The address is in `.dvc/config` (committed); the login (username + token) is in `.dvc/config.local`, which is never committed.

Why DagsHub? Free DVC remote storage with a simple login, and it connects to the GitHub repo, so it shows the code and the DVC-tracked files together in one page.

Why are the data files not on GitHub? `dvc add` puts them in `.gitignore`. GitHub only has the pointer files (e.g. `red_yolo26n.pt.dvc`); the real files are on DagsHub. That is the point of DVC.

What does `dvc add` do? Hashes the file or folder, copies it into the cache, writes the `.dvc` pointer file, and adds the real file to `.gitignore`. Used for raw inputs that no pipeline stage produces (our datasets and released models).

Why didn't you `dvc add` the pipeline outputs (`data/prepared`, `models/model.pt`)? Because they are declared as `outs` in `dvc.yaml`. DVC tracks stage outputs automatically and records their hashes in `dvc.lock`; `dvc add` is only for files that come from outside the pipeline.

What does `dvc dag show`? The pipeline as a graph (DAG = directed acyclic graph): `red.dvc` → `prepare` → `train` → `evaluate`, with the released model also feeding `train`. "Acyclic" means no loops, so there is always a valid order to run the stages.

What does `dvc init` do? Turns a Git repo into a DVC project: creates `.dvc/` with `config`, the cache folder and `.dvcignore`. It is the first DVC command, like `git init` for Git.

What is `dvc checkout`? Restores the workspace files to match the current pointers / `dvc.lock`, using the local cache. `dvc pull` = download from the remote (`dvc fetch`) + `dvc checkout`. We use it after switching Git branches: `git checkout green` changes `dvc.lock` to the green run, then `dvc checkout` swaps `models/model.pt` and `data/prepared` to the green versions.

Why do you have a `green` branch? One branch per laser: `main` has `dataset: red`, `green` has `dataset: green`, each with its own `dvc.lock` and metrics committed. It is the usual Git + DVC way to keep experiments side by side: switching is `git checkout <branch>` + `dvc checkout` (seconds, from the cache, no rerun), and `dvc metrics diff main green` / `dvc params diff main green` compare them without switching. Both branches are on GitHub, and their DVC files on DagsHub.

What is the difference between `git checkout` and `dvc checkout`? `git checkout` changes the files Git tracks (code, `params.yaml`, `dvc.lock`, `.dvc` pointers). It does not touch the big files, so right after it `dvc status` shows `data/prepared` and `models/model.pt` as modified. `dvc checkout` then makes the big files match the new pointers.

When you switched back from green to red, why did prepare and train not rerun? DVC's run cache remembers the outputs of every past run of a stage for a given set of inputs. Red with these exact inputs had been run before, so DVC restored the outputs from the cache ("Stage is cached - skipping run") instead of recomputing.

What happens if you change the code (e.g. `src/evaluate.py`)?

Each script is a dependency of its stage, so its hash changes and that stage (and later ones) become out of date. Changing `src/evaluate.py` reruns only evaluate.

What are `metrics` and `plots` in `dvc.yaml`? Special outputs DVC can display and compare. `metrics/metrics.json` → `dvc metrics show / dvc metrics diff`. `metrics/plots/` (PR curve, F1 curve, confusion matrix) and the per-epoch training logs → `dvc plots show`. They use `cache: false`, so they are stored in Git and visible on GitHub.

What is `.dvcignore`? Like `.gitignore`, but for DVC. We use it to ignore the `labels.cache` files Ultralytics writes next to the labels; otherwise DVC would think the prepared dataset changed after every evaluation.

What is `.gitattributes` for? It keeps Unix (LF) line endings on every OS. DVC hashes the exact bytes of the scripts; on Windows, Git would otherwise convert them to CRLF on checkout, change their hashes and make a fresh clone report the stages as changed.

What is `${dataset}` in `dvc.yaml`? Templating: DVC fills it from `params.yaml`. With `dataset: red`, the prepare stage depends on `data/raw/red` and train on `models/released/red_yolo26n.pt`; with `green`, on the green ones.

What is the difference between `dvc repro` and just running the scripts? Running the scripts always redoes everything and records nothing. `dvc repro` runs them in the right order, skips what hasn't changed, and records the hashes of every input and output in `dvc.lock`, so the exact result is versioned and reproducible.

3. The problem and the data

What problem does the project solve? Detecting a laser pointer dot (red or green) in camera frames of a projector screen. It is the input system for an interactive projector game: a phone camera films the screen, the model finds the dot, and the game uses it as a pointer.

Where does the data come from? Our own videos of the projected screen (with and without a laser), cut into 1920×1080 frames. Red: 4 videos → 1,949 frames. Green: 8 clips → 1,069 frames (labelled on Roboflow).

How big is the dataset? Red: 1,748 images (train 1,484 / valid 48 / test 216), about 224 MB. Green: 965 images (train 820 / valid 25 / test 120), about 85 MB. Small enough for fast training and for DVC storage, as the course requires.

What are the labels? YOLO format, one class (`laser`): a text file per image with `0 cx cy w h` (box centre and size, relative to the image, 0–1). Frames with no laser have an empty file (negative examples). Red train: 907 laser / 577 no laser; green train: 519 / 301.

Why include frames without a laser? So the model learns *not* to fire on bright spots, reflections or game graphics. False alarms move the pointer in the game, so negatives matter as much as positives.

How did you split the data, and how did you avoid leakage?

Frames were split in time blocks (whole seconds / groups of consecutive frames), and frames too close in time to a valid or test frame were removed from training (175 red, 104 green).

Neighbouring video frames are almost identical, so without this the test set would leak into training and the scores would be inflated.

What preprocessing does the pipeline do? The `prepare` stage (1) validates every label line (5 values, class 0, box values within 0–1) and stops the pipeline on broken data, (2) resizes frames larger than `prepare.max_side`, and (3) writes `data.yaml` with relative paths so it works on any machine. The heavy data work (frame extraction, splitting, pseudo-labelling) was done in the semi-supervised project and is the input dataset.

Why do you keep full-size frames instead of resizing? We measured it: resizing to 1280 px (`max_side: 1280`) lowered red test mAP50-95 from 0.815 to 0.731. The dot is only a few pixels wide, so shrinking the image makes the box less precise. The default `max_side: 1920` keeps the original frames.

4. The model and semi-supervised learning

Which model do you use? YOLO26n (Ultralytics, nano size): about 2.4 million parameters, 5.2 GFLOPs, 5.5 MB file. One class, input size 1280 px.

Why a nano model? The game runs on a laptop with only an Intel integrated GPU, so the model must be small and fast. The dot is tiny, so a high input resolution (1280) matters more than a bigger network. The course also asks for appropriate model complexity.

What is semi-supervised learning, and how did you use it?

Training with a small labelled set plus a large unlabelled set. Steps:

1. Hand-label a small set (red: 150 train frames; green: 92).
2. Train a large **teacher** (YOLO26l, 1280 px, 150 epochs) on it.
3. The teacher labels the unlabelled pool (pseudo-labels): confidence ≥ 0.5 → laser, < 0.2 → no laser, in between → left out (red) or checked by a human (green). The teacher was also retrained once on its own confident labels (self-training).
4. Train the small **student** (YOLO26n, 1280 px, 100 epochs) on hand + pseudo-labels.
5. Test only on hand-labelled frames the models never saw.

Why semi-supervised instead of labelling everything? Labelling is the slow, expensive part. With about 150 hand-labelled frames, the teacher produced labels for 1,334 more (red). For green we checked this: the teacher's laser / no-laser decision was wrong on 0 of 724 frames whose real labels we had hidden, and its boxes had precision and recall 0.90 at IoU 0.5.

Why a large teacher and a small student? The large model learns best from few labels; the small model is what we can run on the laptop. The student learns from the teacher's labels and matches it: red student test mAP50 0.994, same as the teacher, with a better mAP50-95 (0.815 vs 0.771).

Training settings? From `yolo26n.pt` , 100 epochs, batch 16, image size 1280, seed 0, augmentation: horizontal and vertical flips, $\pm 10^\circ$ rotation, scale 0.3, HSV colour changes, mosaic (off for the last 10 epochs).

Why do you keep the last epoch and not the "best" one? "Best" is chosen on the validation set, which is tiny (red 48 frames / 22 lasers; green 25 frames / 11 lasers), so it is very noisy. Picking a checkpoint on it would be picking noise. We keep the last epoch and report the test set as the main result.

Why doesn't the pipeline's train stage train? Training takes several hours on an RTX 3090. By default (`train.mode: released`) the stage uses our trained semi-supervised student. Setting `train.mode: train` retrains the student step with the same settings; DVC sees the parameter change and reruns train and evaluate. It does not redo the teacher and pseudo-labelling: their result is the dataset in `data/raw` .

Where was the model trained? On an RTX 3090 (24 GB), PyTorch with CUDA, Ultralytics 8.4.63.

5. Results and metrics

model	test frames	precision	recall	mAP50	mAP50-95	image accuracy
Red YOLO26n	216	0.973	0.991	0.994	0.815	97.7 % (1 false alarm, 4 misses)
Green YOLO26n	120	0.833	0.830	0.700	0.289	99.2 % (0 false alarms, 1 miss)

What does precision mean here? Of all boxes the model predicted, the share that really are the laser. Low precision = false alarms.

Recall? Of all real laser dots, the share the model found. Low recall = missed dots.

mAP50? Mean average precision when a predicted box counts as correct if it overlaps the real box by at least 50 % ($\text{IoU} \geq 0.5$). It summarises precision and recall over all confidence thresholds.

mAP50-95? The same, averaged over stricter overlap thresholds from 0.50 to 0.95. It measures how *precisely* the box fits.

What is IoU? Intersection over Union: the overlap area of the predicted and real box divided by their combined area. 1 = perfect fit.

What is image-level accuracy? For each test frame: did the model correctly decide "laser" or "no laser" (any box with confidence ≥ 0.25)? This is what matters for the game.

Why is green's mAP lower than red's, but its accuracy higher?

The green dot is smaller and often overexposed, so its boxes are harder to place exactly, which the strict box metrics punish. But deciding whether a laser is present is almost always right (119 of 120). The green test set is also small (120 frames), so each mistake moves the numbers a lot.

Why is mAP50-95 low in general? The dot is a few pixels wide. A box one or two pixels off already fails the high IoU thresholds, even though the dot is found in the right place.

Why are the validation numbers so different from test (green valid mAP50 0.225)? The validation set has only 25 frames with 11 lasers, so one or two mistakes change the score enormously. That is why the test set (120 frames) is the main result.

Do your pipeline results match the original ones? On the PC (NVIDIA GPU), exactly: `dvc repro` reproduces 0.973 / 0.991 / 0.994 / 0.815 and 97.7 % for red, and 0.833 / 0.830 / 0.700 / 0.289 and 99.2 % for green.

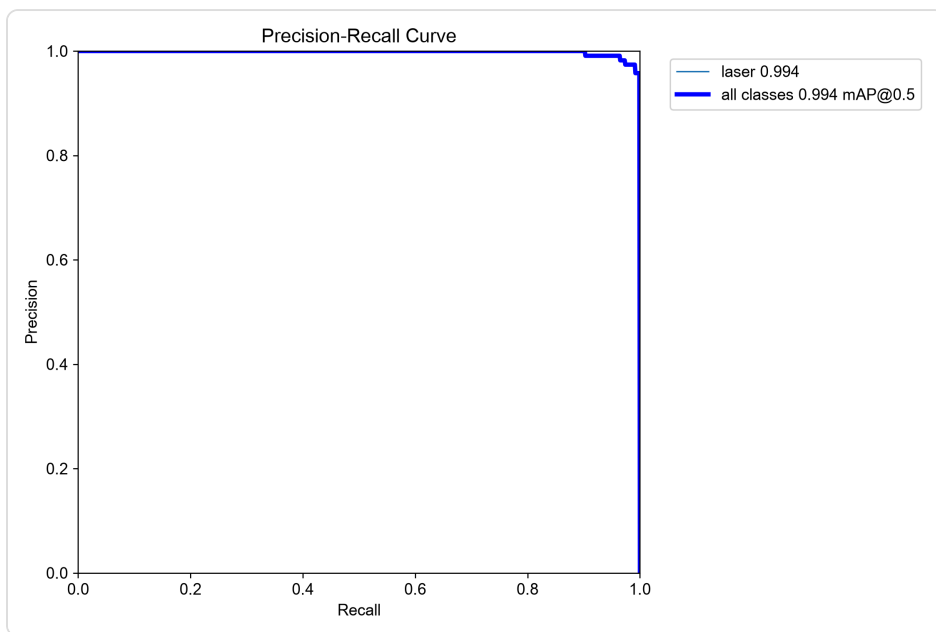
Results on the Mac (Apple GPU, `mps`) — these are the numbers now committed in `metrics/metrics.json` on `main` (red) and `green` :

model	precision	recall	mAP50	mAP50-95	image accuracy
Red, PC (CUDA)	0.973	0.991	0.994	0.815	97.7 % (1 false alarm, 4 misses)
Red, Mac (MPS)	0.974	0.991	0.994	0.813	98.1 % (1 false alarm, 3 misses)
Green, PC (CUDA)	0.833	0.830	0.700	0.289	99.2 % (0 false alarms, 1 miss)
Green, Mac (MPS)	0.833	0.833	0.700	0.286	99.2 % (0 false alarms, 1 miss)

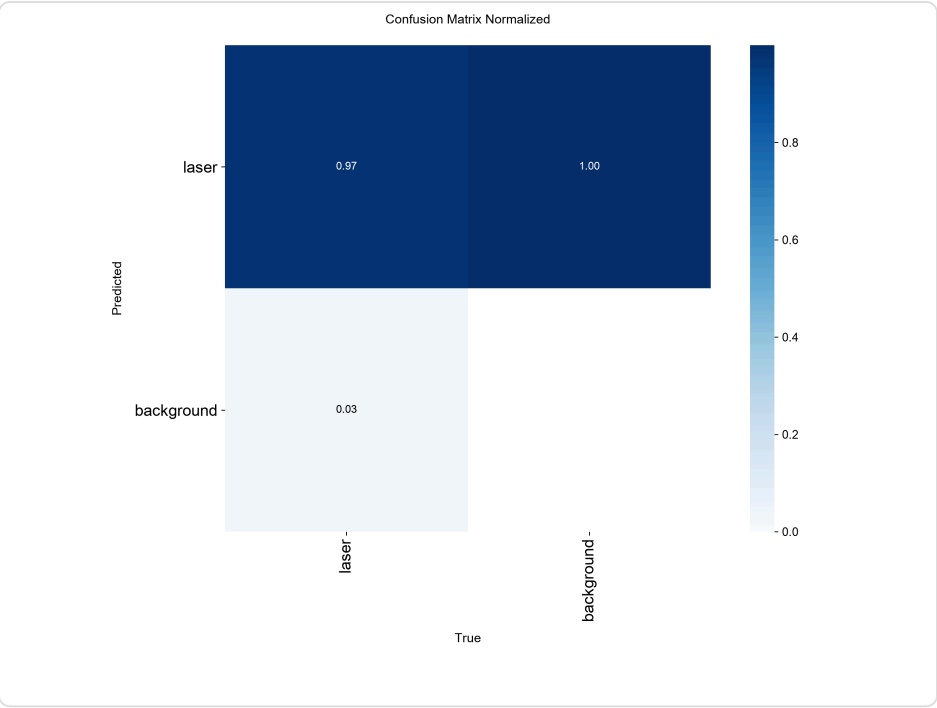
Why are the Mac numbers not exactly the same? Same data, same model file, same code and parameters, but a different GPU. NVIDIA and Apple GPUs do floating-point maths in a slightly different order, so confidences and box coordinates differ in the last digits. mAP50-95 moves in the third decimal, and one red test frame whose confidence was right at the 0.25 threshold flipped from "miss" to

"found". This is hardware-level noise, not a change in the experiment; DVC guarantees the same inputs, the hardware decides the last bits of the output.

Show the curves. `metrics/plots/` (tracked in Git, `dvc plots show`), red on `main`:

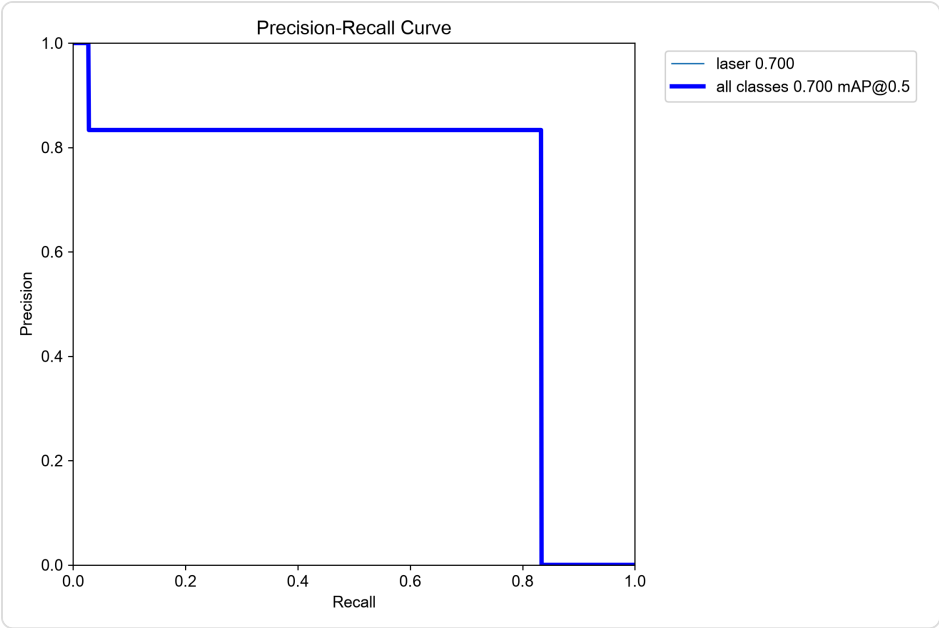


Red precision-recall curve

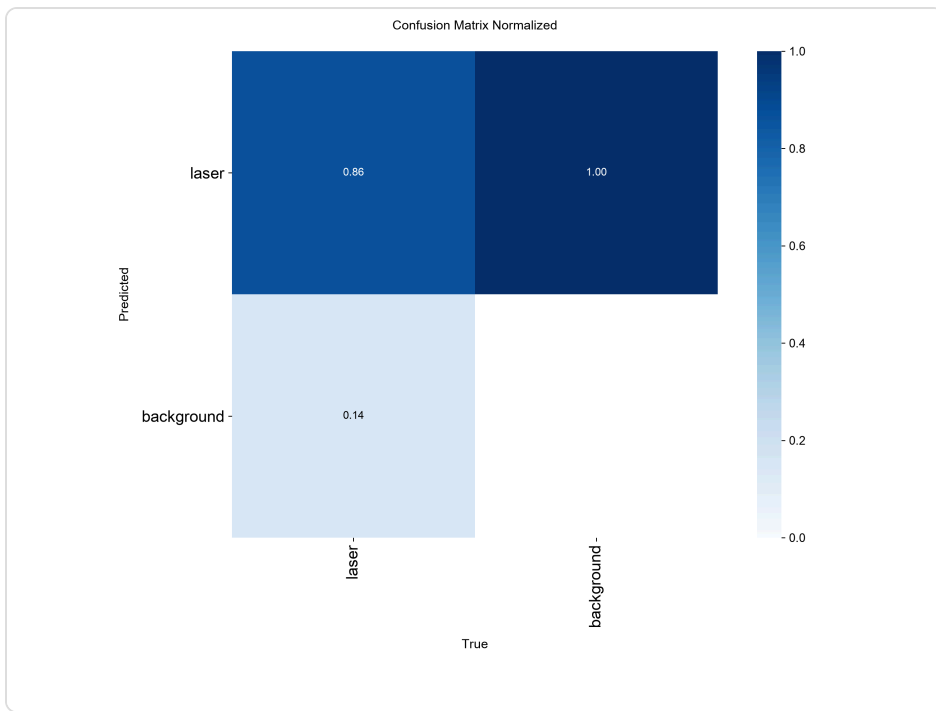


Red confusion matrix

Green, on the `green` branch:



Green precision-recall curve



Green confusion matrix

What does the confidence threshold do? A box is kept only if the model's confidence is at least that value. Higher threshold → fewer false alarms but more misses; lower → the opposite. `evaluate.conf` in `params.yaml` (0.25) sets it for the image-level accuracy.

Why does live detection use 0.80 for red but 0.25 for green? On a laptop webcam pointed at a normal room, the red model at 0.25 gave many false alarms (red / bright objects that look like a dot); green gave none. We measured the trade-off on the red test set before raising it:

red threshold	false alarms (103 empty frames)	misses (113 lasers)
0.25	1	3
0.5 – 0.7	1	4
0.80	0	8
0.90	0	17

0.80 removes the false alarms for 5 extra misses (93 % recall). The values live in `params.yaml` → `detect.conf` ; the pipeline's `evaluate.conf` stays 0.25 so the reported metrics are unchanged.

6. Project structure and repo

Explain the project structure.

```

• • •
data/raw/{red,green}      datasets (DVC; pointers red.dvc / green.
dvc in Git)
data/prepare/            output of prepare (DVC)
models/released/         semi-supervised YOLO26n models (DVC; .dvc
                           pointers in Git)
models/model.pt          output of train (DVC)
src/prepare.py, train.py, evaluate.py  the three stages
src/device.py            picks the device: NVIDIA GPU -> Apple GP
                           U (mps) -> CPU
main.py                  live detection with the pipeline's model
(models/model.pt)
detect.py                live detection with a chosen released mo
del (--laser red|green|both)
metrics/                 metrics.json, data_stats.json, plots (Gi
t)
reports/released/        summaries and per-epoch training logs of
the trained models
docs/                    screenshots for these notes and the PDF
versions
dvc.yaml, dvc.lock, params.yaml, requirements.txt, README.md

```

What is in Git and what is in DVC? Git: code, `dvc.yaml`, `dvc.lock`, `params.yaml`, `.dvc` pointers, metrics, plots, README. DVC (DagsHub): the datasets, the models, `data/prepared`, `models/model.pt`.

What does `requirements.txt` contain? `ultralytics==8.4.63` (the version the models were trained and evaluated with), `dvc`, OpenCV, PyYAML, NumPy. PyTorch is installed separately (see README): the CUDA build on the PC, the normal build on the Mac (it already supports the Apple GPU).

Does it need a GPU? No. `src/device.py` picks the NVIDIA GPU on the PC, the Apple GPU (`mps`) on the Mac, and the CPU anywhere else; train and evaluate use it. A GPU only makes it faster. `src/device.py` is a dependency of train and evaluate in `dvc.yaml`, so changing it reruns them.

7. Bonus components (not implemented)

If asked about Feast, online learning, River or drift detection: **we did not implement them**, following the PDF's note not to add technologies without a real purpose. If asked which would fit:

- **Drift detection** fits best. Lighting, camera exposure or the projector can differ from the training frames (*data drift*), and detection quality drops. River's ADWIN detector could watch the stream of detection confidences in the game and warn when its average shifts, prompting recalibration or collecting new frames to retrain.

- **Online learning** doesn't fit: a YOLO detector can't be updated frame by frame like an incremental model; it is retrained in batches.
 - **Feast / feature store** doesn't fit: feature stores are for tabular features shared between training and serving; our input is raw images.
-

8. Limitations and future work

- Small test sets (216 red, 120 green) and tiny validation sets, so the numbers have noticeable uncertainty.
 - The data comes from one room / one projector setup; other lighting or screens may need more data (that is where drift detection and retraining would help).
 - The pseudo-labels can contain mistakes; for green we measured them (0 wrong laser / no-laser decisions out of 724), for red we could not, because the hidden labels did not exist.
 - The default pipeline reuses the trained model; full retraining takes hours.
 - On a laptop webcam in a normal room the red model gives false alarms at the default 0.25 threshold. That is exactly the data drift described in section 7: the training frames are all of a projector screen. Raising the live threshold to 0.80 hides it; the real fix is to add frames from the new camera / room (including empty ones) and retrain.
-

9. Running it on the Mac and live detection

What did you change to run it on the MacBook? Only two things.

(1) `device=0` (NVIDIA only) in `train / evaluate` became `best_device()` from `src/device.py`: NVIDIA GPU → Apple GPU (mps) → CPU. (2) The python.org Python had no SSL certificates, so `dvc pull` failed with `CERTIFICATE_VERIFY_FAILED`; pointing `SSL_CERT_FILE` at `certifi`'s certificate file fixed it (no code change).

Why did DVC rerun train and evaluate after that change? The scripts are dependencies of their stages, and `src/device.py` was added as a new dependency in `dvc.yaml`. Their hashes changed, so `dvc status` showed `train` and `evaluate` out of date (prepare was untouched, so it didn't rerun).

What is `main.py`? Live detection with the pipeline's own output. It loads `models/model.pt` (what the train stage produced for the current `dataset`), reads its settings from `params.yaml` → `detect:` (source, image size, per-laser confidence) and shows the camera feed with the dot boxed. The command is always `python main.py`; which laser it detects is decided by DVC (the branch / `dataset` parameter), not by a flag.

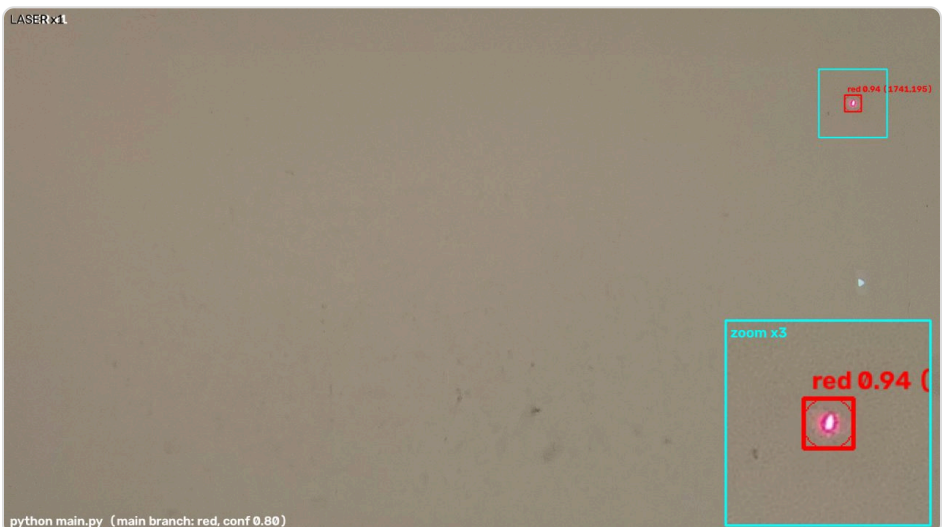
What sources does it support? A webcam (`source: 0`, or `1` for an iPhone via Continuity Camera), an RTSP or HTTP stream (`rtsp://user:pass@ip:554/...`), a video file or a single image. For live streams a background thread keeps only the newest frame, so if the model is slower than the camera it skips frames instead of falling further and further behind.

How fast is it on the Mac? YOLO26n on the M4's GPU: about 48 frames per second at 1280 px, 64 at 960 px, 73 at 640 px. 1280 is the default because the models were trained at 1280 and the dot is tiny.

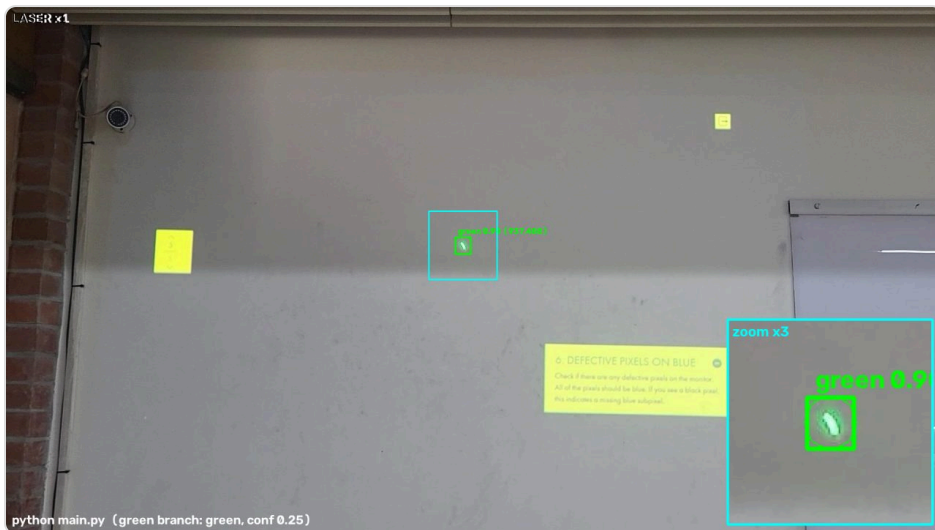
How do you make sure `main.py` uses the right model? It checks three things agree before loading the model: `params.yaml`'s `dataset`, the dataset recorded for the train stage in `dvc.lock`, and the MD5 hash of the real `models/model.pt` against the hash in `dvc.lock`. Changing `dataset` without `dvc repro`, or switching branch without `dvc checkout`, stops it with the command to run. It is the same hash check DVC itself does.

Why doesn't editing `detect:` in `params.yaml` make the pipeline out of date? A stage only depends on the parameters listed under its `params:` in `dvc.yaml`. No stage lists `detect`, so DVC ignores it; only `main.py` reads it.

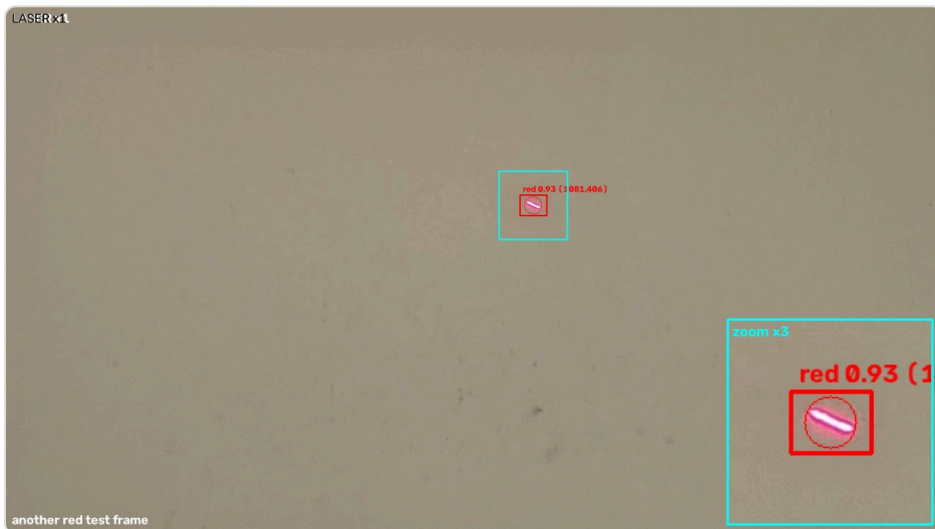
What does the output look like?



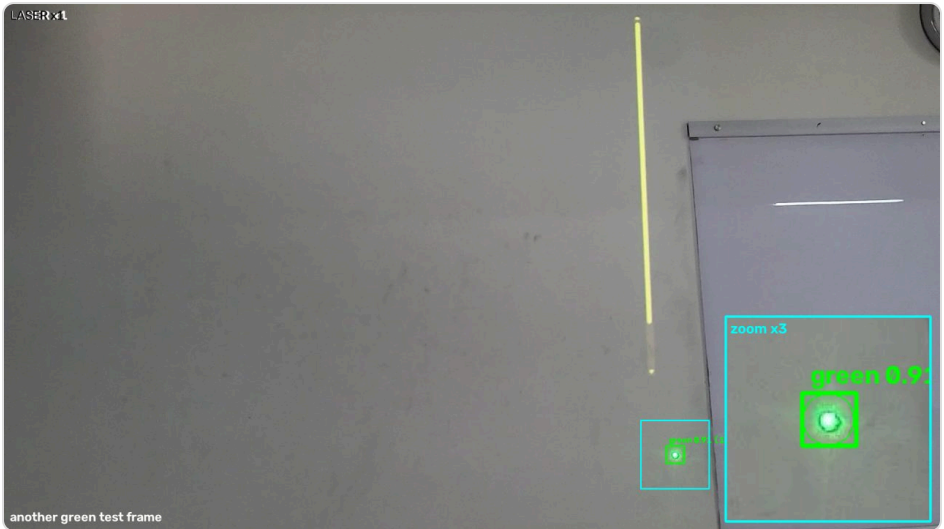
main.py on the main branch: red dot found with confidence 0.94



main.py on the green branch: green dot found with confidence 0.90



Another red test frame



Bright fluorescent tube ignored, only the dot found



Frame without a laser: nothing detected

The cyan box marks the area shown magnified $\times 3$ in the corner (the dot is only about 20 pixels wide in a 1920×1080 frame). The top-left text is the status line the live window shows (`LASER x1` / `no laser` , plus FPS for a live stream).